

Claude Certified Architect (Foundations) — Last-Minute Revision Guide

Raj Rajarathinam Senior Architect - Cognizant Organized for maximum exam performance. Read straight through or jump to the section you need.

TABLE OF CONTENTS

- 1. [Exam Overview](#)
- 2. [High-Probability Topics by Domain](#)
- 3. [Things to Memorize](#)
- 4. [Decision Tables](#)
- 5. [Exam Traps](#)
- 6. [Scenario Questions](#)
- 7. [Best Practices](#)
- 8. [Architecture Patterns](#)
- 9. [Common Mistakes](#)
- 10. [Quick Definitions \(Alphabetical\)](#)
- 11. [Top 100 Facts](#)
- 12. [Final 10-Minute Review](#)

1. EXAM OVERVIEW

Format & Structure

Item	Detail
Total domains	5
Total task statements	30
Practice questions in study guide	230+
Diagnostic test	25 questions (5 per domain, ~15 min)
Per-lesson practice questions	1 scenario + 4–5 exam-trap questions

Domain Weights

#	Domain	Weight	Modules
1	Agentic Architecture & Orchestration	27%	7
2	Tool Design & MCP Integration	18%	5
3	Claude Code Configuration & Workflows	20%	6
4	Prompt Engineering & Structured Output	20%	6

#	Domain	Weight	Modules
5	Context Management & Reliability	15%	6

What the Exam Emphasizes

- **Decision-making** over memorization: "Which approach is correct given this constraint?"
- **Anti-patterns** are as important as correct patterns — exam presents tempting wrong answers
- **Production consequences** of design choices (what breaks if you use approach X?)
- **Deterministic vs probabilistic** enforcement — a core recurring theme across all domains
- Questions structured as: scenario → 4 options → correct answer + explanations of why others fail

Question Structure Pattern

Every question presents:

1. A realistic production scenario with a specific constraint or failure mode
2. Four options — typically 1 correct + 1 tempting-but-wrong + 2 clearly wrong
3. The tempting wrong answer is usually "more prompting" or "a more complex architecture"

2. HIGH-PROBABILITY TOPICS BY DOMAIN

Domain 1: Agentic Architecture & Orchestration (27%)

Core Concepts

- **Agentic loop lifecycle**: Send → Inspect `stop_reason` → Execute tool OR terminate
- **stop_reason values**: `"tool_use"` = continue; `"end_turn"` = terminate; anything else = investigate
- **Hub-and-spoke orchestration**: coordinator at center, subagents never talk to each other directly
- **Context isolation**: subagents start with ONLY what coordinator explicitly passes — no inherited history
- **Enforcement spectrum**: hooks (deterministic) vs prompts (probabilistic)
- **Task decomposition**: fixed sequential pipelines vs dynamic adaptive decomposition
- **Session management**: `--resume` vs `fork_session` vs fresh start + summary injection
- **Attention dilution**: architectural problem requiring multi-pass architecture (not bigger models)

Architecture Patterns

- Coordinator spawns subagents via `Task` tool (alias: `Agent` in Claude Code v2.1.63+)
- Coordinator's `allowedTools` MUST include `"Task"` — binary gate
- Parallel spawning: emit multiple `Task` calls in **single coordinator response** (not sequential)
- Multi-pass architecture: per-item local analysis passes + cross-item integration pass
- Prerequisite gates: code checks that physically block downstream tools

Key APIs/Config

- `stop_reason` field on Messages API response
- `SubagentStart` hook (fires when subagent spawned)
- `SubagentStop` hook (fires when subagent returns results)

- **PreToolUse** hook (before execution — blocks/modifies)
- **PostToolUse** hook (after execution — transforms results)

Important Terminology

- **Agentic loop**: deterministic 4-step control flow (not a retry loop, not a chatbot turn)
- **Coordinator**: hub agent that decomposes, routes, aggregates
- **Subagent**: spoke agent handling a specialized task
- **Prerequisite gate**: code-level check that blocks downstream tool until condition met
- **Attention dilution**: uneven attention allocation when processing many items in single pass
- **fork_session**: independent branch from shared baseline (divergent exploration)
- **--resume**: continue specific named session (continuation, same line of investigation)

Production Best Practices

- Always check **stop_reason**, never text content, for loop termination
 - Use **SubagentStart** for rate-limiting and context validation
 - Use **SubagentStop** for schema validation and sensitive data stripping
 - Programmatic gates for financial/security/compliance — never prompt-only
 - Multi-pass reviews for large codebases (per-file + cross-file)
 - Fresh session + summary injection when files change between sessions
-

Domain 2: Tool Design & MCP Integration (18%)

Core Concepts

- **Tool description quality**: primary mechanism LLMs use for selection — 5 required elements
- **Structured error responses**: 4 categories — transient, validation, business, permission
- **Tool overload**: 4–5 tools per agent maximum for reliable selection
- **tool_choice modes**: "auto" / "any" / forced named tool
- **MCP scoping**: project-level (**.mcp.json**) vs user-level (~/.**claude.json**)
- **Built-in tools**: Read, Write, Edit, Bash, Grep, Glob — correct tool for each task
- **Grep vs Glob**: contents vs paths

Architecture Patterns

- Scoped cross-role tools: give agent constrained version of another role's tool to avoid routing overhead
- Tool splitting: one generic tool → multiple specific tools
- MCP resources: expose content catalogs without exploratory tool calls
- Error propagation: structured error context with partial results — never silent suppression

Key Terminology

- **isError flag**: MCP flag telling model tool execution failed, enabling recovery reasoning
- **isRetryable**: whether a retry can ever succeed (true for transient/validation, false for business/permission)
- **MCP resource**: read-only content catalog (schemas, issue lists) vs tool (acts on data)

- **Scoped cross-role tool:** constrained version of another agent's capability given directly to reduce routing hops
- **Barrel file (index.ts):** re-exports from multiple modules; trace by module name not individual function

5 Elements of a Production-Grade Tool Description

1. What the tool does (primary purpose)
2. What inputs it expects (types, formats, constraints)
3. Example queries it handles well
4. Edge cases and limitations
5. Explicit "use THIS not THAT" boundaries

Error Categories (memorize)

Category	Retryable	Recovery
Transient	Yes (as-is)	Retry after delay
Validation	Yes (after fix)	Fix input, retry
Business	No	Escalate, alternate workflow
Permission	No	Different credentials/escalation

Domain 3: Claude Code Configuration & Workflows (20%)

Core Concepts

- **CLAUDE.md hierarchy:** user (~/.claude/) → project (.claude/ or root) → directory-level
- **CLAUDE.md vs settings.json:** CLAUDE.md = concatenated guidance (probabilistic); settings.json = strict precedence (deterministic)
- **Skills/commands:** .claude/skills/ (canonical) or .claude/commands/ (alias)
- **Path-specific rules:** .claude/rules/ with YAML paths frontmatter
- **Plan Mode vs direct execution:** ambiguity vs well-defined scope
- **The -p flag:** non-interactive print mode — critical for CI

Key Distinctions

- CLAUDE.md files **concatenated**, not overridden — conflicts resolve arbitrarily
- settings.json has **strict precedence:** managed policy > local > project > user
- CLAUDE.md delivered as a **user message**, NOT system prompt — no strict compliance guarantee
- Skills = on-demand task workflows; CLAUDE.md = always-loaded universal standards
- **context:** fork in skills frontmatter = isolated sub-agent (keeps main context clean)
- **allowed-tools** in skills = pre-approve without prompts (does NOT restrict other tools)

CLI Flags to Know Cold

Flag	Effect
------	--------

Flag	Effect
<code>-p / --print</code>	Non-interactive; processes prompt, prints to stdout, exits
<code>--output-format json</code>	Force JSON output
<code>--json-schema '<schema>'</code>	Validate final output against JSON Schema
<code>--system-prompt "<text>"</code>	Replace entire default system prompt
<code>--append-system-prompt "<text>"</code>	Append to default system prompt
<code>--max-turns <n></code>	Cap number of agentic turns
<code>--bare</code>	Skip hooks, skills, MCP, CLAUDE.md (fast scripted runs)
<code>--permission-mode <mode></code>	Set permission mode
<code>--allowedTools</code>	Tools that run without permission prompt
<code>--disallowedTools</code>	Deny rules; bare tool name removes tool
<code>-c / --continue</code>	Resume most recent conversation
<code>-r / --resume <id></code>	Resume specific session

CI/CD Key Facts

- **Without -p**: Claude Code hangs indefinitely waiting for interactive input
 - **Batch API**: 50% cost savings, up to 24-hour processing, NO latency SLA — unsuitable for blocking workflows
 - **Self-review** in same session is less effective than independent instance
 - **Include prior review findings** in subsequent runs to avoid duplicate comments
-

Domain 4: Prompt Engineering & Structured Output (20%)

Core Concepts

- **Vague instructions** ("be conservative", "high confidence") → consistent failures
- **Concrete examples** beat prose instructions for consistent output
- **Few-shot**: 2–4 examples with reasoning (not just input-output pairs)
- **tool_use with JSON schemas**: eliminates syntax errors; does NOT prevent semantic errors
- **Validation-retry loops**: effective for format/structural errors; NOT effective for absent information
- **Batch API**: 50% savings, 24-hour window, no SLA, no multi-turn tool calling
- **Multi-pass review**: per-file passes + cross-file integration; independent instance for review

tool_choice Modes

Mode	Behavior	Use When
"auto" (default)	Model decides: tool or text	General operation

Mode	Behavior	Use When
"any"	Must call a tool, chooses which	Guaranteed structured output with multiple schemas
{"type": "tool", "name": "X"}	Must call specific tool X	Mandatory first step

Schema Design Rules

- Make fields **optional/nullable** when source may lack information → prevents fabrication
- Use "unclear" enum value for genuinely ambiguous source
- Use "other" + detail string for extensible categorisation
- Include `calculated_total` + `stated_total` for automatic discrepancy detection
- Schema guarantees **structure** only — semantic validation requires additional logic

When to Use Each Technique

Problem	Technique
Inconsistent formatting across invocations	Concrete input/output examples
Inconsistent judgment on ambiguous cases	Few-shot with reasoning
Complex transformations with edge cases	Test-driven iteration
Unfamiliar domain (might miss considerations)	Interview pattern
Malformed JSON output	<code>tool_use</code> with JSON schemas
Fabricated values for missing fields	Optional/nullable schema fields
Extraction sum doesn't match total	Validation-retry loop
False positives in one category destroy trust	Disable that category while fixing

Domain 5: Context Management & Reliability (15%)

Core Concepts

- **Progressive summarisation trap**: destroys transactional data (amounts, IDs, dates)
- **Persistent case facts blocks**: extracted transactional data outside summarised history
- **"Lost in the middle"**: key findings at beginning + explicit section headers (structural fix)
- **Tool result trimming**: trim to relevant fields before entering conversation history
- **Context degradation**: verbose exploration output buries earlier discoveries (not a token limit problem)
- **Scratchpad files**: persist key findings outside conversation context
- **Escalation triggers**: explicit human request, policy gap, inability to progress (NOT sentiment)
- **Aggregate metrics trap**: 97% overall can hide 40% accuracy on specific document types
- **Stratified random sampling**: sample HIGH-confidence items too (novel errors only caught here)
- **Information provenance**: claim-source mappings must survive synthesis step

Escalation Rules (absolute)

1. **Explicit human request** → escalate IMMEDIATELY, no investigation first
2. **Policy gap** (policy is silent) → escalate; Policy violation (documented "no") → do NOT escalate
3. **Inability to progress** (actually tried and failed) → escalate; "Might not be able to" → NOT sufficient
4. **Sentiment** (frustrated customer) → NOT a valid escalation trigger
5. **Low confidence score** → NOT a reliable escalation trigger

Context Hierarchy (most to least token-efficient)

1. Persistent case facts blocks (never summarised)
2. Structured tool result trimming
3. Incremental codebase discovery (Grep → Read → Grep)
4. Scratchpad files for exploration findings
5. Subagent delegation (context isolation, not just parallelisation)
6. `/compact` proactively during extended sessions

3. THINGS TO MEMORIZE

`stop_reason` Values

- `"tool_use"` → loop continues
- `"end_turn"` → loop terminates
- `"pause_turn"` → continue long-running server-tool turn
- `"max_tokens"` → hit token limit
- `"stop_sequence"` → stop sequence encountered
- `"refusal"` → model declined on 200 response
- **Rule:** treat anything other than `"end_turn"` as "not finished, investigate"

Hook Types & Timing

Hook	When	Use For
<code>PreToolUse</code>	Before tool executes	Block, modify, redirect
<code>PostToolUse</code>	After tool executes, before model processes	Transform results, normalize data
<code>SubagentStart</code>	When subagent spawned via Task/Agent	Rate limiting, logging, context validation
<code>SubagentStop</code>	When subagent returns to coordinator	Schema validation, sensitive data stripping

MCP Configuration Levels

Location	Scope	Version Controlled	Use For
<code>.mcp.json</code> (project root)	Project	Yes	Team-wide servers
<code>~/.claude.json</code> (home dir)	User	No	Personal/experimental

CLAUDE.md Configuration Levels

Location	Scope	Shared	Use For
~/.claude/CLAUDE.md	User	No	Personal preferences
.claude/CLAUDE.md or root CLAUDE.md	Project	Yes (git)	Team standards
Subdirectory CLAUDE.md	Directory	Yes (git)	Package-specific
CLAUDE.local.md (any level)	Same level	No (gitignored)	Personal per-repo quirks

Skills vs CLAUDE.md

Feature	Skills	CLAUDE.md
Loading	On-demand	Always
Purpose	Task workflows	Universal standards
Location (canonical)	.claude/skills/	.claude/CLAUDE.md
Location (alias/alt)	.claude/commands/	Root CLAUDE.md

Built-in Tool Selection Rules

Tool	Use For
Grep	Searching file contents (find function calls, imports, text patterns)
Glob	Matching file paths/names (.test.tsx, **/config.)
Edit	Targeted modifications (unique text match)
Read + Write	Last resort when Edit cannot disambiguate
Bash	Shell-only operations not covered by dedicated tools

Batch API Facts

- **50%** cost savings vs synchronous
- **Up to 24 hours** processing (no latency SLA)
- **No multi-turn tool calling** within single batch request
- Use `custom_id` fields for request/response correlation
- Unsuitable for blocking workflows (pre-merge checks)

Session Management

Option	Use When
--resume <session>	Prior context valid, files unchanged
fork_session	Divergent exploration from shared baseline
Fresh start + summary injection	Files modified, stale context, degraded history

Decomposition Pattern Selection

Task Characteristics	Pattern
Steps known in advance, structured input	Fixed sequential pipeline
Open-ended, unknown scope, legacy exploration	Dynamic adaptive decomposition

Key Numbers

- **4–5 tools** per agent for reliable selection
- **2–4 few-shot examples** (fewer = no pattern; more = diminishing returns)
- Programmatic enforcement required for **>0% failure rate** in financial/security/compliance
- Prompt-based guidance: ~90–95% compliance (non-zero failure rate)
- **97%** aggregate accuracy can hide **40–60%** error rates on specific document types
- Batch API processes up to 24h; design around worst case, not best case

4. DECISION TABLES

Prompt vs Programmatic Enforcement

Requirement	Use	Avoid	Why
Financial operation (refund, transfer)	Programmatic hook/gate	Prompt instructions	Single failure = financial loss
Security operation (identity verification)	Programmatic hook/gate	Prompt instructions	Single bypass = security breach
Compliance (AML, regulatory)	Programmatic hook/gate	Prompt instructions	Single miss = legal penalties
Formatting preference	Prompt	Hooks	Low stakes, occasional deviation acceptable
Style guidelines	Prompt	Hooks	Low stakes
100% compliance required	Hook	Prompt	Prompts are probabilistic
~95% compliance acceptable	Prompt	Hooks	Acceptable deviation, simpler

Tool vs MCP vs Built-in

Requirement	Use	Avoid	Why
Standard Jira/GitHub/Slack integration	Community MCP server	Building custom	Already maintained, tested
Proprietary internal system	Custom MCP server	Community server	No community server exists
File content search	Grep	Glob, Bash grep	Purpose-built, correct permissions

Requirement	Use	Avoid	Why
File path/name matching	Glob	Grep	Purpose-built for path patterns
Targeted file modification	Edit	Read+Write	Context-efficient, safer
Full file rewrite	Write	Edit	When Edit cannot disambiguate
Team-wide tool config	<code>.mcp.json</code>	<code>~/.claude.json</code>	Version-controlled, shared
Personal experimental server	<code>~/.claude.json</code>	<code>.mcp.json</code>	Not shared with team

Agent vs Workflow

Requirement	Use	Avoid	Why
Predictable, structured steps known upfront	Fixed sequential pipeline	Dynamic decomposition	Consistent, debuggable
Open-ended investigation, unknown scope	Dynamic adaptive decomposition	Fixed pipeline	Cannot adapt to discoveries
Multi-file code review	Multi-pass (per-file + cross-file)	Single-pass	Attention dilution
Divergent approaches from same baseline	<code>fork_session</code>	<code>--resume</code>	Independent branches
Continue same investigation	<code>--resume</code>	<code>fork_session</code>	Preserves full context
Files changed since last session	Fresh start + summary	<code>--resume</code>	Stale context causes contradictions

Hook vs Prompt

Requirement	Use	Why
Block action if policy violated	<code>PreToolUse</code> hook	Runs before execution; can physically prevent
Transform tool output before model sees it	<code>PostToolUse</code> hook	Data normalization happens every time
100% enforcement of business rule	Hook	Deterministic
Preferred behavior, occasional deviation OK	Prompt	Probabilistic but simpler
Validate subagent output schema	<code>SubagentStop</code> hook	After subagent returns results

Requirement	Use	Why
Rate-limit subagent spawning	<code>SubagentStart</code> hook	Before subagent executes

CRITICAL: PostToolUse hooks CANNOT block actions — action already occurred. Use PreToolUse to block.

Context Window vs Retrieval

Requirement	Use	Avoid	Why
Transactional facts (amounts, IDs, dates)	Persistent case facts block (always in context)	Progressive summarisation	Summarisation destroys precision
Large verbose tool results	Trim to relevant fields	Full result in context	Silent context budget killer
Codebase exploration	Incremental (Grep→Read→Grep)	All files upfront	Context budget killer
Key findings during long exploration	Scratchpad files	Context only	Immune to context degradation
Subagent verbose output	Subagent delegation (context isolation)	Main agent context	Keeps main context clean
Extended session context quality	<code>/compact</code> proactively	Wait for limit	Proactive beats reactive

Cache vs Recompute

Scenario	Use	Why
Stable system prompt + dynamic user message	Prompt caching with <code>cache_control</code> breakpoint	Reuse processed prefix at fraction of cost
Static content before dynamic	Cache breakpoint after static block	If dynamic before static → no cache hits
Ephemeral cache	<code>{"type": "ephemeral"}</code>	Lasts ~5 min since last use
Bursts of similar requests	Prompt caching	Multiple requests reuse cache
Hours-apart requests	Don't rely on ephemeral cache	Cache expires after ~5 min inactivity

Plan Mode vs Direct Execution

Scenario	Use	Why
Architectural restructuring	Plan mode	Multiple valid approaches, risk of rework

Scenario	Use	Why
Library migration (many files)	Plan mode → Direct execution	Understand dependencies before touching files
Multiple valid implementation approaches	Plan mode	Need design alignment first
Codebase exploration needed	Plan mode + Explore subagent	Keep main context clean
Single-file bug fix with clear stack trace	Direct execution	Problem, location, and solution all clear
One function validation check	Direct execution	Well-scoped, no ambiguity
Config value update	Direct execution	Known fix, known location
Stated-upfront complexity	Plan mode immediately	Don't wait for complexity to emerge

Single Agent vs Multi-Agent

Requirement	Use	Avoid	Why
Parallel independent research tasks	Multi-agent (parallel spawn)	Sequential single agent	Parallelism reduces latency
Task requires more context than one agent has	Multi-agent with delegation	Single large-context agent	Context isolation per subagent
Simple sequential task	Single agent	Multi-agent	Overhead not justified
Coverage of multiple specialized domains	Multi-agent (specialized)	One generalist agent	Tool overload degrades selection
18+ tools needed	Multi-agent (4–5 each)	One agent with 18 tools	Selection reliability degrades

Human Review vs Automation

Scenario	Use	Why
Below calibrated confidence threshold	Human review	Model uncertainty validated against ground truth
Above calibrated confidence threshold	Automate	Validated accuracy justifies automation
Novel document type (no calibration data)	Human review	Cannot set valid thresholds without validation
High-confidence items (for sampling)	Stratified random sample	Novel errors only caught here

Scenario	Use	Why
Ambiguous/contradictory source document	Human review	Model cannot resolve; annotate conflict
Pre-merge CI checks	Real-time (synchronous)	Blocking workflow, must not wait 24h
Overnight technical debt reports	Batch API	Latency-tolerant, 50% savings

Sync API vs Batch API

Workflow	API	Why
Pre-merge checks (developer waits)	Synchronous	Blocking; no latency SLA acceptable
Overnight technical debt reports	Batch	Latency-tolerant, 50% savings
Weekly code audits	Batch	Scheduled, not time-sensitive
Nightly test generation	Batch	Results reviewed next morning
Multi-turn tool calling needed	Synchronous	Batch doesn't support it
Real-time code assistance	Synchronous	Interactive, user-blocking

5. EXAM TRAPS

These are the highest-value section. Each trap is a tempting wrong answer on the exam.

Domain 1: Agentic Architecture Traps

- TRAP:** Using `response.content[0].type == "text"` to detect loop completion **Why wrong:** Claude can return text AND tool_use simultaneously. Text presence ≠ completion. **Correct:** Check `stop_reason == "end_turn"`.
- TRAP:** Arbitrary iteration caps as the PRIMARY stopping mechanism **Why wrong:** Caps cut off useful work OR run unnecessary iterations. Use `stop_reason`. **Correct:** Caps are acceptable safety nets only; `stop_reason` is the authoritative signal.
- TRAP:** Setting `tool_choice: "any"` to prevent Claude returning text **Why wrong:** Forces tool use even when agent is finished → infinite loop.
- TRAP:** Blaming downstream subagents for coverage gaps **Why wrong:** Subagents research what they're assigned. Trace incomplete scope to coordinator decomposition. **Rule:** Incomplete scope = coordinator's decomposition problem. Incomplete depth = subagent problem.
- TRAP:** Assuming subagents inherit coordinator's conversation history **Why wrong:** Subagents have completely isolated context. Everything must be explicitly passed.
- TRAP:** Adding more subagents to fix a decomposition problem **Why wrong:** More subagents with equally narrow assignments won't help. Fix coordinator decomposition.

TRAP: Direct inter-subagent communication for efficiency **Why wrong:** Breaks observability, error handling, and controlled information flow.

TRAP: Enhanced system prompt to fix 8% compliance failure rate for financial operations **Why wrong:** Better prompt reduces to 3-4% but never 0%. Financial needs programmatic gate.

TRAP: Recommending `--resume` after files have been modified **Why wrong:** Preserves stale tool results. Agent reasons from outdated content → contradictory advice.

TRAP: Using `fork_session` to handle stale context **Why wrong:** Fork branches from existing session which still contains stale tool results.

TRAP: Suggesting larger context model to fix attention dilution **Why wrong:** Architectural problem, not model capability. Multi-pass architecture is the fix.

TRAP: PostToolUse hooks to BLOCK policy-violating actions **Why wrong:** PostToolUse runs AFTER execution — action already occurred. Use PreToolUse to block.

TRAP: Routing classifiers to fix per-agent compliance issues **Why wrong:** Classifiers handle routing, not per-agent workflow enforcement.

Domain 2: Tool Design Traps

TRAP: Few-shot examples as first step to fix tool misrouting **Why wrong:** Adds token overhead without addressing root cause. Fix tool descriptions first.

TRAP: Building a routing classifier as first step to fix misrouting **Why wrong:** Over-engineered. Bypasses LLM's natural language understanding.

TRAP: Retrying when tool returns empty result from successful query **Why wrong:** Empty result from successful query = no data exists. Retry produces same result. **Critical distinction:** Access failure (timeout/auth) → consider retry. Valid empty result → accept it.

TRAP: Generic error messages without structured metadata **Why wrong:** Agent cannot distinguish transient failures from business rule violations.

TRAP: Treating business errors as retryable **Why wrong:** Policy violations never resolve through retry. Same result every time.

TRAP: Silently suppressing subagent errors by returning empty results as success **Why wrong:** Coordinator believes search succeeded and won't retry or try alternatives.

TRAP: Routing all simple verifications through coordinator for "correctness" **Why wrong:** 2-3 extra hops per request, up to 40% more latency. Scoped cross-role tool for simple cases.

TRAP: Using Glob to find function callers **Why wrong:** Glob matches paths, not contents. Cannot search inside files.

TRAP: Using Grep to find files by extension **Why wrong:** Glob is purpose-built for path matching (`.test.tsx`, `**/config.`).

TRAP: Defaulting to Read+Write for every modification **Why wrong:** Burns context tokens. Edit is faster and more context-efficient.

TRAP: Jumping straight from non-unique Edit failure to Read+Write **Why wrong:** Documented recovery: widen `old_string` or use `replace_all: true` first.

TRAP: Building custom MCP server for standard Jira integration **Why wrong:** Community servers exist for standard integrations.

TRAP: Committing credentials directly in `.mcp.json` **Why wrong:** Credentials in version control. Use `${ENV_VAR}` syntax.

Domain 3: Claude Code Configuration Traps

TRAP: New team member gets inconsistent results even after cloning same repo **Why wrong** (if you blame the repo): Instructions are in user-level `~/.claude/CLAUDE.md`, not project-level. **Correct:** Move to `.claude/CLAUDE.md` (project-level, shared via git).

TRAP: "More specific scope wins" or "user-level overrides project-level" in CLAUDE.md **Why wrong:** CLAUDE.md files are CONCATENATED, not override hierarchy. Conflicts may resolve arbitrarily.

TRAP: `/memory` triggers configuration loading **Why wrong:** `/memory` is diagnostic (shows what's loaded). Configuration loads automatically by location.

TRAP: Directory-level CLAUDE.md for conventions that apply across many directories **Why wrong:** Only applies to one directory. For cross-directory file types, use path-specific rules with glob.

TRAP: File-type-specific conventions in root CLAUDE.md **Why wrong:** Loads every session regardless. Terraform conventions consume tokens during React work. **Correct:** Path-specific rules in `.claude/rules/` with `paths` frontmatter.

TRAP: Task-specific workflows in CLAUDE.md **Why wrong:** CLAUDE.md for always-loaded universal standards. Task-specific belongs in skills.

TRAP: CI pipeline hangs — fix is `CLAUDE_HEADLESS=true` **Why wrong:** `CLAUDE_HEADLESS=true` doesn't exist. Fix is the `-p` flag.

TRAP: Same-session self-review as effective as independent review **Why wrong:** Same session retains reasoning context. Less likely to question own decisions.

TRAP: Batch API for pre-merge CI checks **Why wrong:** Up to 24-hour processing, no latency SLA. Pre-merge is blocking. Use synchronous.

TRAP: Plan mode for a single-file bug fix with clear stack trace **Why wrong:** Problem, location, and solution are all clear. Direct execution, no overhead needed.

Domain 4: Prompt Engineering Traps

TRAP: "Be conservative" or "only report high-confidence findings" as review criteria **Why wrong:** Vague. No actionable decision boundary. Use explicit categorical criteria.

TRAP: Confidence thresholds to fix false positive problems **Why wrong:** LLM confidence is poorly calibrated. Often confident about wrong findings. **Correct:** Explicit criteria with code examples. Confidence useful only for routing, not fixing FPs.

TRAP: Keeping all review categories active while iterating on high-false-positive category **Why wrong:** High FP rate in one category destroys trust in ALL categories. Disable while fixing.

TRAP: "Add more detailed instructions" when formatting is inconsistent across invocations **Why wrong:** More prose still relies on interpretation. Concrete examples eliminate interpretation.

TRAP: Few-shot examples without reasoning only teach literal pattern-matching **Correct:** Include reasoning in examples so model generalizes to novel patterns.

TRAP: Making all JSON schema fields required to ensure completeness **Why wrong:** Required fields pressure model to fabricate when information absent. **Correct:** Optional/nullable fields allow honest `null`.

TRAP: Believing `tool_use` with JSON schemas prevents ALL errors **Why wrong:** Eliminates syntax errors only. Semantic errors (sum discrepancies, field placement) still occur.

TRAP: `tool_choice: "auto"` when guaranteed structured output required **Why wrong:** Model may choose to return text instead. Use `"any"` to guarantee a tool call.

TRAP: Assuming retries always fix extraction failures **Why wrong:** Retries fix format/structural errors. Cannot produce information absent from source.

TRAP: Switching all workflows to batch for 50% cost savings **Why wrong:** Blocking workflows must remain synchronous. Batch has no latency SLA.

TRAP: Larger context window model to fix attention dilution in multi-file reviews **Why wrong:** Attention quality problem, not context size. Focused per-file passes are the fix.

TRAP: Uncalibrated confidence scores for automated routing **Why wrong:** Raw self-reported confidence poorly calibrated. Must validate against labelled sets.

Domain 5: Context Management Traps

TRAP: Progressive summarisation is safe for customer support conversations **Why wrong:** Systematically destroys numerical values (amounts, IDs, dates, order numbers). **Correct:** Persistent case facts block extracted outside summarised history.

TRAP: "Lost in the middle" solved by telling model to pay equal attention **Why wrong:** Structural problem, not instruction problem. Key findings must be at beginning structurally.

TRAP: Keep full tool results because "model might need them later" **Why wrong:** Untrimmed verbose results exhaust token budget across turns.

TRAP: Selective truncation of conversation history **Why wrong:** API is stateless. Selective truncation breaks conversational coherence.

TRAP: Increasing context window to solve context degradation in exploration **Why wrong:** Not a token limit problem. Verbose output still fills window and buries discoveries.

TRAP: Subagent delegation only benefits parallelisation **Why wrong:** Primary benefit is context ISOLATION — keeps main agent's context clean.

TRAP: `/compact` only when hitting context limits **Correct:** Use proactively during extended sessions to maintain context quality.

TRAP: Sentiment-based escalation **Why wrong:** Frustration $\neq$ case complexity. Furious customer with simple issue = resolve autonomously.

TRAP: Self-reported confidence scores as reliable escalation trigger **Why wrong:** Incorrectly confident on hard cases, uncertain on easy ones — exact wrong behavior.

TRAP: Attempting to resolve before honouring explicit human request **Absolute rule:** Explicit human request → escalate immediately, no investigation first.

TRAP: Selecting from ambiguous customer matches using most recent/most active heuristic **Why wrong:** Privacy violations, incorrect actions. Ask for additional identifiers.

TRAP: Using aggregate accuracy (97%) to justify automating high-confidence extractions **Why wrong:** 97% overall can hide 40-60% on specific document types.

TRAP: Only sampling low-confidence extractions for quality control **Why wrong:** High-confidence items automated. Novel errors only caught by stratified sampling of high-confidence items.

TRAP: Selecting most recent source when two credible sources conflict **Why wrong:** Arbitrarily destroys one value. Annotate both with attribution.

TRAP: Synthesis agent paraphrasing without preserving claim-source mappings **Why wrong:** Attribution dies at synthesis step unless explicitly required to preserve mappings.

6. SCENARIO QUESTIONS

Scenario 1: Loop Termination

Scenario: An agentic system prematurely terminates. Code checks `response.content[0].type == "text"` to determine completion. Claude is returning both text and a `tool_use` block. **Correct Answer:** Replace content-type check with `stop_reason` check. Continue on `"tool_use"`, terminate on `"end_turn"`. **Why others wrong:**

- Iteration cap: addresses runaway loops, not premature termination
- Natural language parsing ("I'm done"): ambiguous
- `tool_choice: "any"`: forces tool use even when agent finished → infinite loop

Scenario 2: Coverage Gaps in Multi-Agent Research

Scenario: Coordinator decomposes "impact of AI on creative industries" into only visual arts subtopics. Output is incomplete — missing music, film, and literature. **Correct Answer:** Root cause is coordinator's task

decomposition. Fix coordinator's scope assignment. **Why others wrong:**

- Blaming downstream subagents: they researched only what was assigned
 - Adding more subagents: equally narrow assignments won't cover missing areas
 - Better prompting of subagents: can't research what they weren't asked to cover
-

Scenario 3: Unsourced Claims in Synthesis

Scenario: Coordinator passes full research findings but no metadata (source URLs, document names, page numbers) to synthesis agent. Output has unsourced claims. **Correct Answer:** Context passing failure — coordinator must pass structured metadata alongside every finding. **Why others wrong:**

- Blaming synthesis agent's prompt: it can only cite sources it was given
 - Retrying synthesis: same result without metadata
 - Adding source requirements to synthesis system prompt: can't cite what it doesn't have
-

Scenario 4: Workflow Compliance Failure (8% Failure Rate)

Scenario: System prompt instructs agent to verify customer identity before issuing refunds. Working 92% of the time; 8% failure already caused wrong-account refunds. **Correct Answer:** Implement a programmatic prerequisite gate: `process_refund` blocked unless `get_customer` has returned verified customer ID. **Why others wrong:**

- Enhanced system prompt: reduces to 3-4% but never 0%
 - Few-shot examples: still probabilistic
 - Routing classifier: handles routing, not per-tool compliance
-

Scenario 5: PreToolUse vs PostToolUse

Scenario: International transfers must pass AML check. Using PostToolUse hook to block transfers that haven't passed AML. **Correct Answer:** Switch to PreToolUse hook. PostToolUse runs AFTER execution — transfer already happened. **Why others wrong:**

- PostToolUse: action already occurred, too late to block
 - System prompt instruction: probabilistic, AML is regulatory requirement
-

Scenario 6: Attention Dilution in 14-File Code Review

Scenario: Agent reviews 14 files in single pass. Files 1-5 get detailed feedback; files 10-14 get shallow feedback. Null pointer bugs and SQL injection missed. Same pattern flagged in file 3, approved in file 11.

Correct Answer: Split into 14 per-file local analysis passes + 1 cross-file integration pass. **Why others wrong:**

- More powerful model with larger context: same attention distribution problem
 - Better prompts: improve average but don't solve fundamental attention allocation
 - Batching files: misses cross-batch issues
-

Scenario 7: Session Resumption After File Changes

Scenario: Agent completed deep analysis of 50-file codebase. 3 files have since been modified. Agent resumes with `--resume` and starts giving contradictory advice. **Correct Answer:** Fresh session + inject summary of prior findings + specify 3 changed files for targeted re-analysis. **Why others wrong:**

- `--resume` and ask agent to re-read modified files: stale tool results remain in history
 - Full re-exploration of 50 files: wasteful when only 3 changed
 - `fork_session`: branches from existing session which still has stale results
-

Scenario 8: Tool Misrouting

Scenario: Two tools `get_customer` and `lookup_order` have near-identical descriptions: "Retrieves customer information" and "Retrieves order details." Agent routes incorrectly 30% of the time. **Correct Answer:** Expand each tool description to include input formats, example queries, edge cases, and explicit "use THIS not THAT" boundaries. **Why others wrong:**

- Few-shot examples as first step: adds overhead without fixing root cause
 - Routing classifier: over-engineered
 - Tool consolidation: valid long-term but more effort than expanding descriptions
-

Scenario 9: Access Failure vs Empty Result

Scenario: Customer says order #8891 exists but agent says "no orders found." Tool returns `{"results": [], "status": "success"}`. **Correct Answer:** Tool doesn't distinguish access failure (database was DOWN) from valid empty result. Must return structured error on access failure with `isError: true`. **Why others wrong:**

- Retrying empty result: if database was down and subagent silently returned success, coordinator won't retry
 - Better prompting: structured error metadata is a tool implementation problem
-

Scenario 10: New Team Member Issue

Scenario: Developer A's Claude Code follows all team naming conventions. Developer B joins, clones repo, gets inconsistent results. **Correct Answer:** Naming conventions stored in Developer A's user-level `~/.claude/CLAUDE.md` (not version-controlled, not shared). Move to project-level `.claude/CLAUDE.md`. **Why others wrong:**

- CLAUDE.md conflict resolution: not the issue — B has no instructions at all
 - `/memory` command: diagnostic, not activation trigger
 - Directory-level config: wrong level for team-wide API conventions
-

Scenario 11: CI Pipeline Hangs

Scenario: CI job runs `claude "Analyse this PR for security issues"` and hangs indefinitely. **Correct Answer:** Add the `-p` flag: `claude -p "Analyse this PR for security issues"` **Why others wrong:**

- `CLAUDE_HEADLESS=true`: environment variable doesn't exist
 - `--batch`: flag doesn't exist
 - Redirecting stdin from `/dev/null`: doesn't switch to print mode
-

Scenario 12: Escalation with Frustrated Customer

Scenario: Customer expresses strong frustration about late delivery but the issue is straightforward (package delayed, replacement available). **Correct Answer:** Acknowledge frustration and offer resolution. "I understand this is frustrating. I can process your replacement right now." Do NOT escalate. **Why others wrong:**

- Sentiment-based escalation: frustration $\neq$ case complexity
 - Escalating because confidence is low: poorly calibrated signal
-

Scenario 13: Aggregate Accuracy Trap

Scenario: System shows 97% overall accuracy. Team decides to automate all high-confidence extractions. **Correct Answer:** Measure accuracy by document type AND field segment first. 97% overall can hide 40-60% errors on specific document types. **Why others wrong:**

- Automating based on aggregate: volume-weighted average masks per-type performance
 - Sampling only low-confidence: high-confidence novel errors go undetected
-

Scenario 14: Conflicting Sources

Scenario: Source A reports 12% market growth; Source B reports 8% growth. Synthesis agent must handle. **Correct Answer:** Annotate both values with source attribution and publication dates. Let consumer decide. **Why others wrong:**

- Select most recent: arbitrarily destroys one value
 - Average values: creates third number neither source reported
 - Pick more authoritative publisher: discards valid data
-

Scenario 15: Context Degradation in Codebase Exploration

Scenario: During long codebase exploration session, agent starts referring to "typical patterns" instead of the specific class names and file paths it discovered earlier. **Correct Answer:** Have agent maintain scratchpad files recording key findings. Reference scratchpad for subsequent questions. **Why others wrong:**

- Larger context window model: not a token limit problem
 - `/compact`: useful but doesn't persist findings outside conversation
 - `--resume`: doesn't address loss of specific detail
-

7. BEST PRACTICES

Always...

- **Always** check `stop_reason` for loop control, never text content
- **Always** append tool results to conversation history before next iteration
- **Always** route all inter-subagent communication through coordinator
- **Always** explicitly pass context to subagents (they start isolated)
- **Always** use programmatic enforcement for financial/security/compliance operations
- **Always** add `-p` flag when running Claude Code in CI pipelines
- **Always** include structured metadata (source URL, document name, page number) when passing findings between agents
- **Always** make JSON schema fields optional/nullable when source may not have the information
- **Always** honour explicit customer requests for human immediately, no investigation first
- **Always** measure accuracy by document type AND field segment before automating
- **Always** calibrate confidence scores against labelled validation sets before using for routing
- **Always** trim tool results to relevant fields before they enter conversation history
- **Always** place key findings summaries at the BEGINNING of aggregated inputs (lost-in-middle fix)

Prefer...

- **Prefer** expanding tool descriptions over adding routing classifiers for misrouting
- **Prefer** concrete input/output examples over more prose for inconsistent formatting
- **Prefer** few-shot examples with reasoning over examples without reasoning
- **Prefer** community MCP servers over building custom for standard integrations
- **Prefer** `Edit` over `Read` + `Write` for file modifications
- **Prefer** persistent case facts blocks over summarisation for transactional data
- **Prefer** stratified random sampling (including high-confidence items) over sampling only low-confidence
- **Prefer** parallel subagent spawning (single coordinator response) over sequential for independent tasks
- **Prefer** fresh start + summary injection over `--resume` when source files have changed
- **Prefer** `fork_session` over `--resume` when exploring divergent approaches
- **Prefer** project-level CLAUDE.md over user-level for team-wide standards

Use...

- **Use** `PreToolUse` hooks to block policy-violating actions
- **Use** `PostToolUse` hooks to normalize data formats
- **Use** `SubagentStart` for rate-limiting subagent spawning
- **Use** `SubagentStop` for schema validation and sensitive data stripping
- **Use** `tool_choice: "any"` when guaranteed tool call needed (not "auto")
- **Use** `tool_choice: {"type": "tool", "name": "X"}` for mandatory first steps
- **Use** Plan Mode for architectural changes, library migrations, multi-file modifications
- **Use** direct execution for single-file bug fixes with clear stack trace
- **Use** `context: fork` frontmatter for verbose/exploratory skills
- **Use** path-specific rules (`.claude/rules/`) for file-type conventions across many directories
- **Use** Batch API for latency-tolerant overnight/weekly workflows
- **Use** synchronous API for blocking pre-merge checks
- **Use** interview pattern for unfamiliar domains (before implementing)
- **Use** test-driven iteration for complex transformations with many edge cases
- **Use** coverage annotations when synthesis must note gaps (rather than silently omitting)

Avoid...

- **Avoid** arbitrary iteration caps as primary stopping mechanism
- **Avoid** parsing natural language phrases ("I'm done") for loop termination
- **Avoid** checking `content[0].type == "text"` for completion detection
- **Avoid** direct subagent-to-subagent communication (breaks observability)
- **Avoid** giving one agent 18+ tools (selection reliability degrades)
- **Avoid** single-pass reviews for large multi-file codebases
- **Avoid** sentiment-based escalation decisions
- **Avoid** aggregate accuracy metrics to justify automation decisions
- **Avoid** team-wide config in `~/.claude.json` (personal, not version-controlled)
- **Avoid** committing credentials directly in `.mcp.json`
- **Avoid** full re-exploration when only a few files changed (targeted re-analysis)
- **Avoid** "Be conservative" or "Be thorough" as review criteria (too vague)
- **Avoid** Read+Write for every modification (use Edit; only fall back when Edit cannot disambiguate)

Never...

- **Never** use PostToolUse hooks to block actions (action already happened)
- **Never** silently suppress errors by returning empty results as success
- **Never** terminate entire multi-agent pipeline on single subagent failure
- **Never** use prompt instructions as the sole enforcement for financial/compliance operations
- **Never** use `--resume` after source files have been modified (stale context)
- **Never** read all files upfront in codebase exploration (context budget killer)
- **Never** run Claude Code in CI without the `-p` flag
- **Never** use Batch API for blocking workflows (no latency SLA)
- **Never** select from ambiguous customer matches using heuristics (most recent/active)
- **Never** summarise transactional data progressively (destroys precision)
- **Never** place dynamic content before static content in cached prompts
- **Never** route all simple operations through coordinator when scoped cross-role tool handles 85% of cases
- **Never** put task-specific procedures in CLAUDE.md (use skills)
- **Never** trust self-reported model confidence without calibration against ground truth

8. ARCHITECTURE PATTERNS

Pattern 1: Agentic Loop

flowchart TD

```

A[Send request to Claude\nMessages API] --> B{Check stop_reason}
B -->|tool_use| C[Execute tool\nAppend result to history]
C --> A
B -->|end_turn| D[Return final response]
B -->|other| E[Investigate: max_tokens\npause_turn, refusal]

```

Key rules:

- History must include ALL prior tool results
- **stop_reason** is THE authoritative signal
- Tool results appended BEFORE next API call

Pattern 2: Hub-and-Spoke Orchestration

flowchart TD

```

    U[User Request] --> C[Coordinator]
    C -->|Task/Agent tool| S1[Subagent: Search]
    C -->|Task/Agent tool| S2[Subagent: Analysis]
    C -->|Task/Agent tool| S3[Subagent: Synthesis]
    S1 -->|structured results| C
    S2 -->|structured results| C
    S3 -->|structured results| C
    C --> R[Aggregated Response]

    S1 -. never .-> S2
    S2 -. never .-> S3
  
```

Key rules:

- ALL communication through coordinator
- Subagents NEVER communicate directly
- Coordinator's **allowedTools** MUST include "Task" (or "Agent")
- Each subagent starts with only what coordinator explicitly passes

Pattern 3: Prerequisite Gate

```

    process_refund tool call
      ↓
    PreToolUse hook intercepts
      ↓
    Check: has get_customer returned verified ID in this session?
      ↓ YES                                ↓ NO
    Allow execution                        Return error:
                                          "Identity verification required"
                                          (action never executes)
  
```

Key rules:

- Gate is CODE, not a prompt
- Model cannot bypass it
- Use for: financial, security, compliance operations

Pattern 4: Multi-Pass Review Architecture

```
Files: [F1, F2, F3, ... F14]
      ↓
Pass 1: Per-file (parallel)
      [Review F1] [Review F2] ... [Review F14]
      ↓
Pass 2: Cross-file integration
      Input: All per-file findings
      Checks: data flow, pattern consistency, API usage, contradictions
      ↓
      Unified report
```

Key rules:

- Per-file passes = consistent depth (no attention dilution)
- Cross-file pass = separate invocation, not continuation of per-file
- Use independent instances for review (not self-review in same session)

Pattern 5: Tool Error Response Structure

```
{
  "isError": true,
  "errorCategory": "transient|validation|business|permission",
  "isRetryable": true|false,
  "text": "Human-readable message",
  "description": "What was attempted, why it failed",
  "partialResults": [...],
  "alternativeApproaches": [...]
}
```

Valid empty result (NOT an error):

```
{
  "isError": false,
  "text": "Query executed successfully. No matching records found.",
  "resultCount": 0,
  "shouldRetry": false
}
```

Pattern 6: Structured Context Passing (Coordinator → Subagent)

```
{
  "task": "Synthesize findings on renewable energy policy",
  "findings": [
```

```

    {
      "claim": "EU investment reached €200B in 2023",
      "source_url": "https://example.com/eu-report",
      "document_name": "EU Energy Report 2024",
      "page_number": 14,
      "confidence": "high",
      "retrieved_by": "web_search_agent"
    }
  ],
  "output_requirements": {
    "format": "structured_report",
    "include_citations": true,
    "flag_conflicts": true
  }
}

```

Pattern 7: Prompt Caching Layout

```

[SYSTEM PROMPT]
└─ Long stable instructions (static)
└─ Reference documents (static)
    └─ cache_control breakpoint ← here
[USER MESSAGE]
└─ Dynamic user content (volatile, changes every request)

```

Rule: Static content BEFORE cache breakpoint. Dynamic content AFTER. Dynamic before static = no cache hits.

Pattern 8: Validation-Retry Loop

```

extraction = initial_extract(document)
validated = validate(extraction)

if not validated.valid:
    if validated.fixable: # format/structural error
        extraction = retry_with_error(document, extraction, validated.error)
    else: # absent information
        flag_for_human_review(document, validated.error)

```

```

// Retry message includes 3 things:
{
  role: "user",
  content: `Original document:\n${doc}\n\n` +
    `Your extraction:\n${JSON.stringify(failed)}\n\n` +
    `Validation error: ${specificError}`
}

```

Pattern 9: Persistent Case Facts Block

```
{
  "caseFactsBlock": {
    "customerId": "C-4421",
    "conversationSummary": "Customer requesting refund for defective headphones",
    "issues": [{
      "orderId": "#8891",
      "orderDate": "2024-03-03",
      "refundAmount": "$247.83",
      "status": "pending_refund",
      "itemDescription": "Wireless headphones – defective"
    }]
  }
}
```

Included in EVERY prompt. Never summarised. Persists regardless of conversation history state.

Pattern 10: CI Pipeline Integration

```
# Correct non-interactive usage
claude -p \
  --output-format json \
  --json-schema '{"type":"object","properties":{"findings":{"type":"array"}}}' \
  --append-system-prompt "Focus on security vulnerabilities" \
  "Review this PR: $(cat diff.txt)"

# For subsequent runs: include prior findings
claude -p "Review this PR. Prior findings: ${PREVIOUS_FINDINGS}. Report only NEW
or still-unaddressed issues."
```

Pattern 11: MCP Server Configuration

```
// .mcp.json (project-level, version-controlled, team-shared)
{
  "mcpServers": {
    "jira": {
      "command": "npx",
      "args": ["@community/mcp-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}",
        "JIRA_URL": "${JIRA_URL}"
      }
    }
  }
}
```

```

    }
  }
}

```

Each developer sets `JIRA_TOKEN` locally. Config safe to commit. Credentials never in repo.

Pattern 12: Path-Specific Rules

```

# .claude/rules/test-conventions.yaml
---
paths: ["**/*.test.ts", "**/*.test.tsx", "**/*.spec.ts"]
---
# Test conventions
- Use `describe`/`it` block structure
- Mock external dependencies with jest.mock()
- Assert specific error messages, not just error type

```

Loads ONLY when editing test files. Terraform rules don't consume tokens during React work.

9. COMMON MISTAKES

Implementation Mistakes

- **Checking text content instead of `stop_reason`** for loop termination
- **Not appending tool results** to conversation history before next iteration
- **Sequential subagent spawning** when tasks are independent (unnecessary latency)
- **Resuming sessions** after source files modified (stale context contradictions)
- **Missing `-p` flag** in CI commands (pipeline hangs)
- **Committing credentials** to `.mcp.json` instead of using `${ENV_VAR}`
- **Reading all files upfront** instead of incremental Grep → Read → Grep

Design Mistakes

- **Prompt-only enforcement** for financial/compliance/security operations
- **Too many tools per agent** (18 tools → selection degrades; use 4-5 scoped per role)
- **Single-pass review** of large multi-file codebases (attention dilution)
- **User-level config** for team-wide conventions (not version-controlled, not shared)
- **Routing all operations through coordinator** when scoped cross-role tool handles most cases
- **No prerequisite gates** for high-stakes sequential operations
- **Hub-and-spoke violation**: allowing subagents to communicate directly

Prompt Mistakes

- **Vague criteria** ("be conservative", "high confidence") with no actionable boundary
- **More prose** to fix inconsistent formatting instead of concrete examples
- **Keeping problematic categories active** while fixing false positives (destroys trust in all categories)

- **Making schema fields required** when source may not have the information
- **Not including prior review findings** in subsequent CI runs (duplicate comments)
- **Not including reasoning** in few-shot examples (model only pattern-matches, doesn't generalize)

Tool Mistakes

- **Using PostToolUse to block** — runs after execution, too late
- **Not distinguishing access failure from valid empty result** (different recovery actions)
- **Treating business errors as retryable** (policy violations never resolve via retry)
- **Silently suppressing errors** (returning empty success instead of structured error)
- **Generic error messages** without error category or retry guidance
- **Glob for content search** or **Grep for path matching** (backwards)
- **Defaulting to Read+Write** instead of Edit for targeted modifications

MCP Mistakes

- **Building custom MCP server** for standard Jira/GitHub/Slack (community servers exist)
- **Team config in ~/.claude.json** instead of **.mcp.json** (personal, not shared)
- **Sparse MCP tool descriptions** (model defaults to built-in Grep instead of richer MCP search)
- **Leaving MCP resources unused** (causing unnecessary exploratory tool calls)

Context Mistakes

- **Progressive summarisation of transactional data** (destroys amounts, IDs, dates)
- **Placing key findings in middle of aggregated inputs** (lost-in-middle effect)
- **Not trimming verbose tool output** (silent context budget exhaustion)
- **Using --resume with modified files** (stale tool results cause contradictory advice)
- **Not using scratchpad files** during long exploration (context degradation)
- **Using /compact only when hitting limits** instead of proactively)

10. QUICK DEFINITIONS (ALPHABETICAL)

Agentic loop — Deterministic 4-step control flow: send request → check **stop_reason** → execute tools if needed → terminate on **end_turn**. Not a retry loop, not a chatbot turn.

Attention dilution — Uneven attention allocation when processing many items in a single pass; early items get disproportionate attention, later items get shallow analysis. Fixed by multi-pass architecture.

Barrel file (index.ts) — Re-exports from multiple modules; when tracing function usage, search for barrel file's module name, not individual function after re-export.

Batch API — Asynchronous API with 50% cost savings, up to 24-hour processing window, no latency SLA, no multi-turn tool calling. Suitable for latency-tolerant workflows only.

cache_control breakpoint — Marker in API request that tells Claude to cache everything before it. Ephemeral type lasts ~5 minutes since last use. Static content must come before breakpoint.

Case facts block — Structured JSON containing transactional facts (customer ID, order numbers, amounts, dates) included in every prompt, never summarised, persists regardless of conversation history state.

CLAUDE.local.md — Personal configuration file appended after CLAUDE.md at same level; gitignored by convention; for per-repo personal quirks.

CLAUDE.md — Configuration files providing guidance to Claude Code. Concatenated (not overridden) across hierarchy. Delivered as user message, not system prompt — probabilistic compliance.

context: fork frontmatter — Skills frontmatter that runs skill in isolated sub-agent context; verbose output stays in fork, main conversation stays clean.

Coordinator — Hub agent in orchestration pattern; decomposes tasks, selects subagents, routes information, aggregates results. All inter-subagent communication flows through it.

Dynamic adaptive decomposition — Task decomposition pattern that generates subtasks based on discoveries at each step. Best for open-ended investigation with unknown scope.

end_turn — **stop_reason** value indicating Claude has finished and loop should terminate.

fork_session — Creates independent branch from shared analysis baseline. Use for divergent exploration from common starting point.

Fixed sequential pipeline — Task decomposition pattern with steps defined in advance, executing in order. Best for predictable structured tasks (code reviews, document processing).

Glob — Built-in tool for matching file PATHS by naming patterns (*.test.tsx*, ***/config.*). Does not search file contents.

Grep — Built-in tool for searching file CONTENTS by regex pattern. Does not match by filename.

Hook — Deterministic code that intercepts tool calls before or after execution. PreToolUse (before) for blocking; PostToolUse (after) for transformation. 100% enforcement guarantee.

Hub-and-spoke — Multi-agent orchestration topology where all communication flows through central coordinator. Subagents never communicate directly with each other.

isError flag — MCP tool response field; when true, tells model tool execution failed and enables recovery reasoning.

isRetryable — Error response field indicating whether a retry can ever succeed. True for transient/validation errors; false for business/permission errors.

Lost-in-the-middle effect — Models process beginning and end of long inputs reliably; middle content may be missed. Fixed by placing key summaries at beginning and using explicit section headers.

MCP (Model Context Protocol) — Protocol for connecting Claude to external tools and data sources. Configured at project level (*.mcp.json*) or user level (*~/.claude.json*).

MCP resource — Read-only content catalog (database schemas, issue summaries) exposed to agents without exploratory tool calls. Resources = visibility; tools = action.

-p flag — **--print** mode for Claude Code CLI; processes prompt, outputs to stdout, exits non-interactively. Required for CI pipelines.

Path-specific rules — Rule files in `.claude/rules/` with YAML `paths` frontmatter; load only when editing files matching those glob patterns.

Plan Mode — Claude Code mode where Claude reads, analyses, and proposes without modifying files. Use for ambiguous, multi-approach, multi-file tasks.

PostToolUse hook — Runs after tool executes, before model processes result. For data transformation and normalisation. Cannot block actions (too late).

Prerequisite gate — Code-level check that blocks a downstream tool until a required condition is met. Deterministic enforcement pattern for sequential dependencies.

PreToolUse hook — Runs before tool executes. Can block, modify, or redirect calls. For policy enforcement.

Programmatic enforcement — Code-level enforcement (hooks, gates) that physically prevents disallowed actions. Deterministic; 100% compliance guarantee.

Progressive summarisation trap — Danger of summarising long conversations: systematically destroys transactional data (amounts, IDs, dates, order numbers).

Prompt-based guidance — Instructions in system prompt. Works 90-95% of the time. Non-zero failure rate. Probabilistic. Never use for financial/security/compliance requirements.

Scoped cross-role tool — Constrained version of another agent's capability given directly to an agent, to avoid routing overhead for common operations.

Session state manifest — Structured JSON checkpoint capturing explored paths, key findings, and next steps; enables crash recovery without full re-exploration.

Skills — On-demand task-specific workflows in `.claude/skills/` (or `.claude/commands/`). Loaded only when invoked. Support frontmatter for isolation, tool pre-approval, argument hints.

stop_reason — The authoritative signal for agentic loop control. `"tool_use"` = continue; `"end_turn"` = terminate. Never use text content for this determination.

Stratified random sampling — Quality control technique that samples from EACH stratum (document type, confidence band). Must include high-confidence items to catch novel error patterns.

Subagent — Spoke agent in hub-and-spoke orchestration; handles specialized task, receives from coordinator, returns to coordinator. Starts with only what coordinator explicitly passes.

SubagentStart hook — Fires when subagent spawned via Task/Agent tool. Use for rate limiting, logging, context validation.

SubagentStop hook — Fires when subagent finishes and returns results to coordinator. Use for schema validation, sensitive data stripping.

Task tool — Mechanism for spawning subagents from a coordinator. Renamed to `Agent` in Claude Code v2.1.63+; `Task` still works as alias. Coordinator's `allowedTools` must include it.

tool_choice: "any" — Forces model to call a tool (any tool) rather than returning text. Use when guaranteed structured output required with multiple possible schemas.

tool_choice: "auto" — Default; model decides whether to call tool or return text. May choose text — not suitable when structured output guaranteed.

tool_use — **stop_reason** value indicating Claude wants to call a tool; loop must continue with tool execution.

Validation-retry loop — Pattern of sending failed extraction back with original document and specific error message for model to self-correct. Effective for format/structural errors; not for absent information.

11. TOP 100 FACTS

1. **stop_reason == "end_turn"** is the ONLY reliable signal to terminate an agentic loop.
2. **stop_reason == "tool_use"** means continue — execute the tool and loop back.
3. Claude can return text AND tool_use simultaneously; text presence ≠ completion.
4. Tool results MUST be appended to conversation history before the next API call.
5. All subagent communication flows through the coordinator — never direct inter-subagent.
6. Subagents start with ONLY what the coordinator explicitly passes; no inherited history.
7. Coordinator's **allowedTools** MUST include **"Task"** (or **"Agent"**) — binary gate for spawning subagents.
8. Incomplete output scope → trace to coordinator's decomposition, not downstream subagents.
9. Parallel subagent spawning = emit multiple Task calls in a SINGLE coordinator response.
10. **SubagentStart** hook fires when subagent is spawned; use for rate-limiting and validation.
11. **SubagentStop** hook fires when subagent returns results; use for schema validation and data stripping.
12. **PreToolUse** runs BEFORE execution — can block/modify/redirect. Use for policy enforcement.
13. **PostToolUse** runs AFTER execution — can only transform, NOT block. Action already occurred.
14. Hooks are deterministic (100% enforcement); prompts are probabilistic (90-95%).
15. Financial/security/compliance operations ALWAYS require programmatic enforcement (hooks or gates).
16. A prerequisite gate is CODE that blocks downstream tool until condition met — model cannot bypass.
17. Fixed sequential pipelines for predictable structured tasks; dynamic decomposition for open-ended investigation.
18. Attention dilution = per-item + cross-item multi-pass architecture (not bigger models, not better prompts).
19. **--resume** keeps full conversation history; use when prior context is still valid.
20. **fork_session** creates independent branches for divergent exploration from shared baseline.
21. Fresh start + summary injection when files modified since last session (stale context = contradictory advice).
22. Targeted re-analysis: start fresh, inject summary, re-read ONLY the changed files.
23. Tool descriptions are the PRIMARY mechanism for LLM tool selection — 5 elements required.
24. Tool misrouting → expand descriptions FIRST (not classifiers, not few-shot, not consolidation).
25. 4–5 tools per agent maximum for reliable selection.
26. Scoped cross-role tools avoid routing overhead for common cross-domain operations.
27. Business errors are NEVER retryable — same policy violation every time.
28. Transient errors: retry as-is. Validation errors: fix input, then retry. Business/Permission: escalate.
29. **isError: true** tells the model tool execution failed, enabling recovery reasoning.
30. Access failure (database down) vs valid empty result (no data found) → different recovery actions.
31. Silently suppressing errors by returning empty success is the WORST anti-pattern.
32. Structured error context must include: failure type, what was attempted, partial results, alternatives.

33. Grep searches file CONTENTS; Glob matches file PATHS/NAMES.
34. Edit before Read+Write; widen `old_string` or use `replace_all` before falling back to Read+Write.
35. Edit failure is a safety mechanism preventing unintended modifications — not an error to bypass.
36. `.mcp.json` (project root) = version-controlled, team-shared. `~/.claude.json` = personal only.
37. `${VARIABLE_NAME}` syntax in `.mcp.json` keeps credentials out of version control.
38. Community MCP servers for standard integrations (Jira, GitHub, Slack) — build custom only for proprietary.
39. Sparse MCP tool descriptions → model prefers built-in tools (Grep) over richer MCP tools.
40. MCP resources expose content catalogs without exploratory tool calls.
41. CLAUDE.md files are CONCATENATED, not overridden. Conflicts may resolve arbitrarily.
42. settings.json has STRICT precedence: managed policy > local > project > user. Managed always wins.
43. CLAUDE.md is delivered as a user message, NOT system prompt — no strict compliance guarantee.
44. User-level `~/.claude/CLAUDE.md` = personal, not shared. Project-level = version-controlled, shared.
45. New team member getting inconsistent results → conventions in user-level config, not project-level.
46. `/memory` command is DIAGNOSTIC (shows what's loaded), not activation trigger.
47. `CLAUDE.local.md` appended after `CLAUDE.md` at same level; gitignored by convention.
48. `.claude/skills/` is canonical skills location; `.claude/commands/` is backward-compatible alias.
49. `context: fork` frontmatter: skill runs in isolated context. Without it, verbose output pollutes main context.
50. `allowed-tools` in skills frontmatter pre-approves tools without prompts — does NOT restrict other tools.
51. Skills = on-demand task workflows. CLAUDE.md = always-loaded universal standards.
52. Task-specific procedures belong in skills, NOT CLAUDE.md.
53. Path-specific rules in `.claude/rules/` with `paths` frontmatter — load only when editing matching files.
54. Path-specific rules are more token-efficient than root CLAUDE.md for file-type conventions.
55. `**/*.test.tsx` glob catches every test file across entire codebase regardless of directory depth.
56. Plan Mode = safe exploration, no file modification. For ambiguous multi-approach tasks.
57. Direct execution for single-file bug fix with clear stack trace — plan mode adds unnecessary overhead.
58. Decision for plan vs direct = ambiguity, NOT difficulty.
59. When complexity stated upfront in requirements, choose plan mode IMMEDIATELY.
60. The `-p` flag switches Claude Code to print mode (non-interactive). Without it, CI hangs.
61. `CLAUDE_HEADLESS=true` does NOT exist. `--batch` flag does NOT exist for Claude Code CLI.
62. Batch API: 50% cost savings, up to 24-hour processing, NO latency SLA, NO multi-turn tool calling.
63. Pre-merge CI checks = synchronous API (blocking). Technical debt reports = Batch API (latency-tolerant).
64. Same-session self-review is less effective; independent instance evaluates fresh.
65. Including prior review findings in subsequent CI runs prevents duplicate comments eroding trust.
66. CLAUDE.md files in CI behave same as interactive mode — include CI-relevant sections.
67. `--append-system-prompt` when Claude should stay a coding assistant + extra rules.
68. `--system-prompt` (replace) when pipeline needs different identity or permission model.
69. Vague criteria ("be conservative") → no actionable decision boundary → consistent failures.
70. Concrete input/output examples beat prose instructions for consistent formatting.
71. Few-shot: 2–4 examples WITH reasoning. Reasoning enables generalization to novel patterns.
72. False positives in one review category destroy trust in ALL categories — disable while fixing.
73. `tool_use` with JSON schemas eliminates SYNTAX errors; semantic errors still require validation.

74. `tool_choice`: "any" guarantees a tool call; "auto" may return text instead.
 75. Optional/nullable schema fields prevent fabrication for absent information.
 76. "unclear" enum value handles genuinely ambiguous source classification.
 77. Include `calculated_total` and `stated_total` for automatic discrepancy detection.
 78. Retry-with-error-feedback requires 3 parts: original document + failed extraction + specific error.
 79. Retries fix format/structural errors. Retries CANNOT produce information absent from source.
 80. Batch API does NOT support multi-turn tool calling within a single request.
 81. Optimize prompts on 5-10 sample documents BEFORE batch submission.
 82. Multi-pass review: per-file local passes for depth + separate cross-file integration pass.
 83. Confidence-based routing requires calibration using labelled validation sets.
 84. Progressive summarisation destroys transactional data (amounts, IDs, dates, order numbers).
 85. Persistent case facts block: transactional facts outside summarised history, in every prompt.
 86. Lost-in-the-middle: place key findings FIRST + explicit section headers (structural fix).
 87. Trim verbose tool results to relevant fields before they enter conversation history.
 88. API is stateless — each request must include complete conversation history.
 89. Explicit human request → immediate escalation, no investigation first (absolute rule).
 90. Policy gap (silence) → escalate. Policy violation (documented "no") → do NOT escalate.
 91. Sentiment (frustration) is NOT a valid escalation trigger. Complexity is the trigger, not emotion.
 92. LLM self-reported confidence is poorly calibrated — incorrectly confident on hard, uncertain on easy.
 93. Ambiguous customer matching → ask for identifiers; never use heuristics (most recent/active).
 94. 97% aggregate accuracy can hide 40-60% error rates on minority document types.
 95. Stratified random sampling must include HIGH-confidence items to catch novel error patterns.
 96. Context degradation = verbose output burying earlier discoveries (not a token limit problem).
 97. Scratchpad files persist key findings OUTSIDE conversation context — immune to degradation.
 98. Subagent delegation primary benefit = context ISOLATION (not just parallelisation).
 99. `/compact` should be used PROACTIVELY during extended sessions, not only at limits.
 100. Synthesis is the highest-risk step for attribution loss — explicitly require claim-source mapping preservation.
-

12. FINAL 10-MINUTE REVIEW

Read this immediately before the exam. No explanations — only the highest-yield points.

Loop Control

- `stop_reason`: "end_turn" = stop. "tool_use" = continue. Never use text content to decide.
- Checking `content[0].type == "text"` = wrong. Claude can return text + tool_use simultaneously.
- Iteration caps = safety nets only, not primary stopping mechanism.

Orchestration

- All communication through coordinator. Subagents NEVER talk to each other.
- Subagents get ONLY what coordinator explicitly passes (completely isolated context).
- Incomplete scope = coordinator decomposition fault. Not the subagent.
- Spawn subagents in parallel = multiple Task calls in ONE coordinator response.
- `allowedTools` must include "Task" to spawn subagents.

Enforcement

- Financial/security/compliance = PROGRAMMATIC (hooks, gates). Never prompt-only.
- PreToolUse = block/modify BEFORE execution. PostToolUse = transform AFTER (cannot block).
- Hooks = deterministic (100%). Prompts = probabilistic (90-95%).

Claude Code Config

- CLAUDE.md = concatenated (not overridden). settings.json = strict precedence.
- New dev inconsistency = instructions in user-level `~/.claude/CLAUDE.md` not project-level.
- Skills = on-demand. CLAUDE.md = always-loaded. `context: fork` = isolates verbose output.
- Path-specific rules in `.claude/rules/` = token-efficient file-type conventions.
- CI: ALWAYS use `-p` flag. Without it = hangs. `CLAUDE_HEADLESS=true` doesn't exist.
- Batch API = blocking CI is WRONG. Batch has no latency SLA (up to 24 hours).
- Self-review in same session = less effective. Use independent instance.
- Plan Mode = ambiguity. Direct execution = clear scope, known approach.

Tools & MCP

- Grep = contents. Glob = paths. Edit first; Read+Write last resort.
- Edit non-unique match → widen `old_string` OR use `replace_all`. Not immediately Read+Write.
- 4-5 tools max per agent. Scoped cross-role tool for common cross-domain operations.
- `.mcp.json` = team (version-controlled). `~/.claude.json` = personal only.
- Community MCP server first; build custom only for proprietary systems.
- `${ENV_VAR}` in `.mcp.json` — never commit credentials.

Prompt Engineering

- Vague criteria = fail. Concrete examples beat prose for consistent output.
- Few-shot: 2-4 examples WITH reasoning (not just input-output pairs).
- High FP in one category = disable it; trust bleeds across all categories.
- `tool_choice: "any"` = guaranteed tool call. `"auto"` may return text.
- Optional/nullable schema fields = prevent fabrication. Required = pressure to fabricate.
- `tool_use` catches syntax errors only. Semantic errors require validation-retry loop.
- Retry works for format/structural errors. Retry CANNOT produce absent information.

Context Management

- Progressive summarisation destroys transactional data → use persistent case facts block.
- Key findings at BEGINNING of aggregated inputs (lost-in-middle fix).
- Trim tool results to relevant fields before they enter context.
- `--resume` only when files unchanged. Modified files = fresh start + summary injection.
- Context degradation = verbose output problem (not token limit). Scratchpad files fix it.
- Subagent delegation = context ISOLATION (primary benefit, not just parallelisation).
- `/compact` proactively, not just when hitting limits.

Escalation Rules (absolute)

- Explicit human request → escalate IMMEDIATELY, no investigation first.

- Policy GAP → escalate. Policy VIOLATION → do NOT escalate (documented "no" is the answer).
- Sentiment (frustration) = NOT a valid escalation trigger.
- Self-reported confidence = NOT a reliable escalation trigger.

Key Numbers

- 4-5 tools max per agent. 2-4 few-shot examples. 50% Batch API savings. 24h max Batch processing.
- 97% aggregate can hide 40-60% per-segment errors. ~5 min ephemeral cache TTL.

Biggest Distractors

- "Better prompts will fix it" — only for low-stakes, formatting, style. Not for compliance failures.
- "Larger context window will fix attention dilution" — architectural problem, not context size.
- "Batch API for pre-merge checks" — blocking workflow, no latency SLA.
- "PostToolUse to block policy violations" — too late, action already happened.
- "More subagents fix coverage gaps" — fix coordinator decomposition instead.
- "Confidence scores drive automated routing" — only AFTER calibration against labelled sets.
- "Aggregate 97% accuracy = safe to automate" — always segment by document type first.

13. STUDY MATERIAL LINKS

Primary Exam Prep

Resource	URL	Covers
Claude Certified Architect Learning Portal	https://claudecertificationguide.com/learn	All 5 domains — 39 lessons, 230+ practice questions
Quick Reference (cheat sheets)	https://claudecertificationguide.com/learn/quick-reference	Decision rules and exam traps per domain
Glossary	https://claudecertificationguide.com/learn/glossary	All key terms with exam context
Diagnostic Test	https://claudecertificationguide.com/learn/diagnostic	25-question self-assessment with per-domain scoring
Build Exercises	https://claudecertificationguide.com/learn/exercises	39 hands-on exercises (one per lesson)

Official Anthropic Documentation

Resource	URL	Covers
Anthropic Docs Home	https://docs.anthropic.com	Full API and SDK reference

Resource	URL	Covers
Messages API Reference	https://docs.anthropic.com/en/api/messages	Agentic loop, stop_reason , tool_use (Domain 1, 4)
Tool Use Guide	https://docs.anthropic.com/en/docs/build-with-claude/tool-use	Tool schema design, tool_choice modes (Domain 2, 4)
Tool Use Overview	https://docs.anthropic.com/en/docs/tool-use	tool_choice , JSON schemas, structured output
Message Batches API	https://docs.anthropic.com/en/docs/message-batches	Batch API, cost savings, latency (Domain 4)
Prompt Caching	https://docs.anthropic.com/en/docs/prompt-caching	cache_control , ephemeral breakpoints (Domain 5)
Agentic Patterns	https://docs.anthropic.com/en/docs/build-with-claude/agentic-and-multi-agent-frameworks	Multi-agent orchestration, subagents (Domain 1)
Computer Use / Extended Thinking	https://docs.anthropic.com/en/docs/build-with-claude/extended-thinking	Advanced Claude capabilities
Models Overview	https://docs.anthropic.com/en/docs/about-claude/models	Current model IDs and capabilities
Anthropic Cookbook (GitHub)	https://github.com/anthropics/anthropic-cookbook	Practical code examples across all domains

Claude Code (Domain 3)

Resource	URL	Covers
Claude Code Docs	https://docs.anthropic.com/en/docs/claude-code	Full Claude Code reference
Claude Code CLI Reference	https://docs.anthropic.com/en/docs/claude-code/cli-reference	All CLI flags including -p , --resume , --bare
CLAUDE.md Configuration	https://docs.anthropic.com/en/docs/claude-code/memory	CLAUDE.md hierarchy, loading order
Hooks Reference	https://docs.anthropic.com/en/docs/claude-code/hooks	PreToolUse, PostToolUse, SubagentStart, SubagentStop
Skills / Slash Commands	https://docs.anthropic.com/en/docs/claude-code/slash-commands	Skills frontmatter, context: fork , allowed-tools
Settings & Permissions	https://docs.anthropic.com/en/docs/claude-code/settings	settings.json precedence, permission modes

Resource	URL	Covers
CI/CD Integration	https://docs.anthropic.com/en/docs/claude-code/github-actions	CI pipeline usage, <code>-p</code> flag in automation
Claude Code GitHub	https://github.com/anthropics/claude-code	Source and issue tracker

MCP (Model Context Protocol) — Domain 2

Resource	URL	Covers
MCP Official Site	https://modelcontextprotocol.io	Protocol overview, architecture
MCP Specification	https://spec.modelcontextprotocol.io	Full spec — tools, resources, prompts
MCP Docs	https://modelcontextprotocol.io/docs	Server implementation, client configuration
MCP GitHub (official servers)	https://github.com/modelcontextprotocol/servers	Reference server implementations
MCP in Claude Code	https://docs.anthropic.com/en/docs/claude-code/mcp	<code>.mcp.json</code> , <code>\${ENV_VAR}</code> syntax, scoping

Anthropic Blog — Production Patterns

Resource	URL	Covers
Anthropic Blog	https://www.anthropic.com/blog	Research and product announcements
Building Effective Agents	https://www.anthropic.com/research/building-effective-agents	Orchestration patterns, tool use, multi-agent (Domain 1, 2)
Prompt Engineering Guide	https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering	Few-shot, system prompts, structured output (Domain 4)

Community & Additional Practice

Resource	URL	Covers
Anthropic Discord	https://discord.gg/anthropic	Community Q&A, exam discussion
Anthropic GitHub	https://github.com/anthropics	All official repos (claude-code, SDKs, cookbooks)

Resource	URL	Covers
Python SDK	https://github.com/anthropics/anthropic-sdk-python	API client with examples
TypeScript SDK	https://github.com/anthropics/anthropic-sdk-typescript	API client with examples

End of revision guide. Good luck on the exam.